Object Encodings for Dynamically Typed Languages

Author Hidden for Double-Blind Review

Abstract

In dynamically typed programming languages, variables and expressions do not have types associated with them; instead, values have types. This means that selecting the correct machine instructions to operate on data requires run-time dispatch based on the types associated with the values. The simplest encoding strategy is to store every object in a memory heap, where each object is tagged with its type. This approach places significant pressure on both the memory subsystem and the memory allocation system. At the other extreme, the most complex encoding strategies place as many values as possible into immediate representations to reduce this pressure. This paper describes seven different encoding strategies, ranging from all-memory representations to NaN-based encodings and evaluates their relative performance compared with a baseline implementation that uses no encoding. We evaluate their relative performance on two architectures (AArch64 and x86-64) using recursive Fibonacci benchmarks that stress both integer and floating-point arithmetic paths under inlined and non-inlined dispatch. Our results reveal a fundamental performance trade-off: encoding with minimal tag-width achieves the fastest integer arithmetic but must heap-allocate floats, while NaN-boxing stores doubles without the overhead but penalizes integer extractions. Heap-only (*ptr*) encodings incur order-of-magnitude penalties from cache pressure regardless of architecture. This paper also introduces the max encoding, which achieves comparable overall performance to NaN-boxing across both workloads while offering a wider immediate integer range (55-bit versus 47-bit) and direct class identification without memory lookups.

Keywords: Dynamic Types, Object Encoding, Performance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MPLR2026, Brussels, Belgium
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN XXXXXXX
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Author Hidden for Double-Blind Review. 2026. Object Encodings for Dynamically Typed Languages. In *Proceedings of The 23rd International Conference on Managed Programming Languages & Runtimes (MPLR2026).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

For the vast majority of modern von Neumann architecture computers, data (whether stored in memory or registers) is a sequence of bits and the interpretation of those bits is determined by the instructions that are applied to operate on them. So an *integer add* instruction interprets the parameters as 2s complement integers, whereas a *float addition* instruction would interpret the same bits as an IEEE-754 floating-point number.

In statically typed programming languages, the compiler has complete knowledge of the types of operator, and hence the instructions to generate, because every variable and function has a defined type (whether provided explicitly or inferred).

Compilers for dynamically typed languages have no such advance knowledge. While compilers and language systems such as Self[8] and Just-In-Time (JIT) compilers may derive local knowledge of types (using type inference, flow analysis, sampling and other techniques) the ground truth is encoded in the values that are provided and computed. This means that implementers of such languages must choose a mechanism to associate the type with every value.

We have chosen a range of encodings and implemented them as alternatives in a Smalltalk run time system. We have then timed the various encodings in a straightforward implementation of a Fibonacci function evaluated over integer and float domains. While our system is primarily oriented to running Smalltalk, the encodings should have broad applicability. Worth pointing out for other languages is that it *may* be beneficial to use the special tag in the Spur case to encode a value with special importance in that language (cons pairs in Lisp, tables in Lua. Only experimentation can tell if this is worthwhile.

Modern processors fetch instructions, decode them, and put them in the execution pipeline. This means that unconditional branches and calls to fixed addresses have almost no cost as long as the pipeline remains relatively full. Similarly, conditional branches are relatively inexpensive, because of branch prediction hardware and sometimes decoding along more than one potential execution path. What *are* expensive are indirect branches because only after the data is available can the fetch-decode logic start adding to the pipeline.

Pipelines can execute instructions out-of-order and sometimes in parallel, as long as dependencies are observed.

There are many options in terms of dynamically-typed language implementation, from byte interpreters to threaded execution to JIT'ed native code. However the encoding choices will affect all of the implementations, to varying degrees.

Paper outline. Section 2 describes the encodings we considered. Section 3 describes the experiment and the observed results. Section 4 contains some conclusions and future work.

2 Object Encodings

In this section we describe - in some detail - 8 different encodings that have been used or proposed in some dynamic language system. For each, we show a possible bit encoding and describe the properties of the encoding.

[3] provides a comprehensive taxonomy of such mechanisms ranging from memory allocation of all objects, to complex tagging of values. [4] has a detailed description of floating-point encodings.

2.1 Encoding Considerations

Encoding values, particularly for modern architectures, have three considerations:

Determining types. Since operations must be dispatched at runtime based on the types of their operands, the type of each value must be determined before an operation can proceed. If the type information are in memory, this can incurs a potential pipeline stall while data can be loaded.

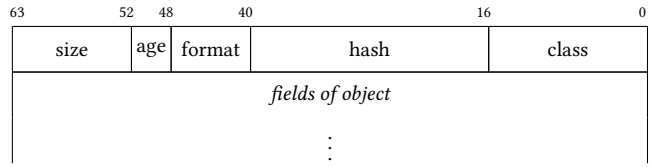
Accessing values. In order to perform operations, the values must be made available to the CPU. Accessing values in memory is an order of magnitude slower than accessing them in registers, and even if the values happen to be in cache they will be several times slower than in registers.

Supporting Memory Management. Dynamically-typed languages invariably have an automatic memory management system, whether reference counting or garbage collection. Objects need to be allocated, and reclaimed when no longer needed. While memory management has improved significantly over the decades, there remains a significant cost to both actions, and the more objects allocated in memory, the greater the cost. We use the term **churn** to refer to the allocation and subsequently collecting in-memory objects.

2.2 Encodings

In an object-oriented language, objects are either mutable or immutable, and this distinction shapes how they can be encoded. Mutable objects must be allocated in memory, and references to them are passed around so that any mutation is visible to all references pointing to the same object. These allocations are typically on a heap and has a header word encoding its class, size, and garbage collection metadata. Immutable objects, by contrast, may be representable without heap allocation. This paper assumes 64-bit words and our encoding model throughout.

For example, our heap object looks like:

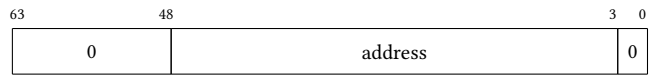


The header contains: `size` is the 12-bit number of words the object occupies; `age` is the 4-bit age of the object; `format` is the 8-bit format of the object; `hash` is the 24-bit identity hash; and `class` is the 16-bit class of the object. It is followed by the fields of the object. The reference to the object is the address of the header word.

There are many possible ways to encode the range of immutable values that arise in normal calculations. In this section we discuss 8 possible encodings that encompass the range from all values being in memory, to treating as many things as immediate as possible. We examine each from the perspective of the three considerations outlined in section 2.1. Table 1 summarizes the important encoding considerations for the encodings discussed below.

2.2.1 Every Object in Memory.

This is the simplest model. Every object - mutable or immutable - is encoded as a word-aligned address:



Here, the low 3 bits are zero as part of the word-aligned address, and the high 16 bits are zero due to hardware limitations.

Determining the value type requires a memory access to get the class from the header, as does accessing the actual value. The result of every operation that produces a value must be heap-allocated, resulting in significant allocation pressure; though most values are short-lived, placing pressure on the garbage collector. Inter-operation with foreign functions is complicated, as pointers must be wrapped in heap-allocated objects before use. Immutable values *nil*, *true*, *false*, and ASCII Characters are normally allocated at fixed addresses to gain some performance by allowing hardcoded memory address comparisons and we assume this convention.

2.2.2 Every Object in Memory with *SmallInteger* Cache.

This is the same encoding as the previous one, with the optimization of statically pre-allocating a set of small integers (in our implementation -5..100). This means that if a result is within this range, we avoid both allocation and collection entirely.

Encoding	Determining type & Accessing Value	Memory Considerations
In Memory	memory access	every created immutable requires allocation
SmallInteger Cache	memory access	common integers avoid allocation
Tag SmallInteger	direct for integer; memory access for all others	integers avoid allocation ; all other immutables require allocation
Tag Pointer	direct for all types	all immutables except *Float* avoid allocation; value field limited to 48 bits
Spur	direct for integer, character, most floats	integer, character, and most float values avoid allocation
Max-Spur	direct for integer, most floats, and all immediates	integer, most floats, and all immediate values (boolean, nil, character, symbol) avoid allocation
NaN Encoding	direct for most immutables	most immutables avoid allocation
Max	direct for most immutables	most immutables avoid allocation

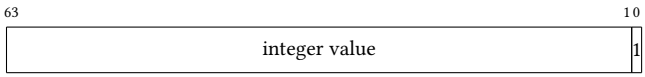
Table 1. Considerations of Encodings

This directly reduces churn for the most frequently created numeric values. It also may get some memory-cache advantage as these values will typically be in cache.

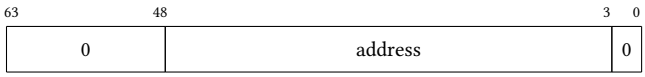
2.2.3 Tag *SmallInteger*.

Early Lisp[9], Self[10], and Smalltalk[2] implementers observed that the vast majority of runtime values are *SmallInteger* objects. Most implementations tag small integers with a 1-bit tag either in the low bit, keeping all other addresses naturally aligned by objects on at least a word boundary. Less commonly, the sign bit can be used instead. This encoding was significant enough that the SPARC architecture provided tagged arithmetic instructions that checked the low bit of the operands to trap on non-integer values.

Here *SmallInteger* values are encoded as:



and everything else remains encoded as the address:



Note that the low bit distinguishes the integer value from the reference.

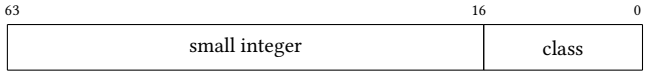
A drawback of this encoding is the loss of one bit of precision; however, most such languages automatically promote to arbitrary-precision integers on overflow, and on 64-bit systems the reduced range is rarely a practical concern.

There is also an alternative to this where the integer case uses a 0 low bit and the pointer case uses a 1 low bit. This is slightly faster for *SmallInteger* at the expense of being slightly slower for everything else.

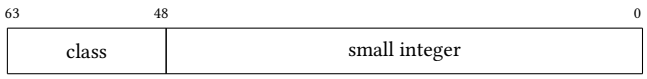
2.2.4 Tag Pointers.

One approach to reducing the cost of type determination is to encode the class directly in every object word. Most 64-bit architectures use 48-bit virtual address spaces, leaving the high 16 bits available for a class tag.

Here *SmallInteger* and other simple immutable values are encoded as:



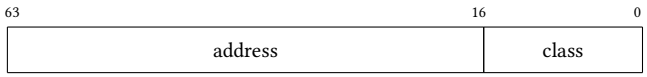
or



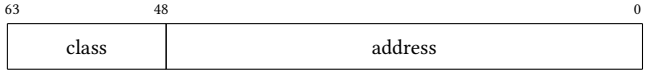
with the class tag occupying the high or low part of the 64-bit word. This is optimized by using class=0 for *SmallInteger* so the value doesn't have to be manipulated before performing addition/subtraction.

This can also encode *Symbol*, *Character*, booleans, and *UndefinedObject* as immediate values.

In our experiment we encode 32-bit floating-point values as immediate values. If the value does not fit the 32-bit floating range, then the value is stored in memory. For heap-allocated objects, the 48-bit address is paired with the class tag:



or

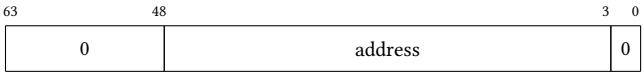


A drawback of this encoding is that we only have 48 bits of precision for *SmallInteger* values. More significantly, the class is embedded in every reference rather than in the object header itself, making *primitiveChangeClassTo:* prohibitively expensive, as every reference to the object must be found and updated.

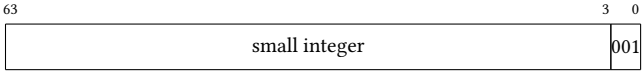
2.2.5 Tag *SmallInteger*, *Character*, *Float* (Spur Encoding).

The Spur[5, 6] encoding extends the 1-bit tag to 3 bits and supporting immediate encodings for Unicode characters and a subset of floating-point values in addition to *SmallInteger*.

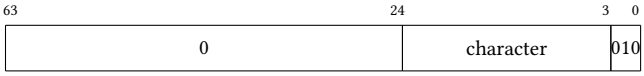
The default encoding for everything not described below remains encoded as the address:



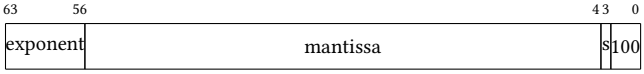
The encoding for *SmallInteger* is:



The encoding for *Character* is:



The encoding for *ImmediateFloat* is:



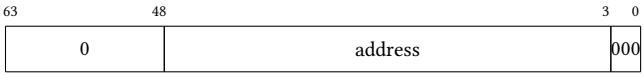
Examining the low 3 bits of an object, an *and* instruction disambiguate the encodings. Only the pointer tag is used directly; all the others require some decoding before use.

The *ImmediateFloat* encoding preserves the full 52-bit mantissa of IEEE-754 double precision, but reduces the exponent range to $\pm 5.87 \times 10^{-39}$ to $\pm 6.80 \times 10^{38}$. The conversion algorithms are detailed in [1] and require 7–8 instructions. Float values outside this exponent range, as well as *inf* and *nan*, are heap-allocated.

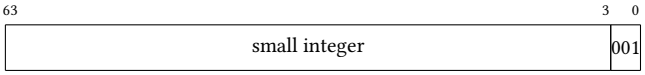
2.2.6 Tag *SmallInteger*, other immediate values, *Float* (MaxSpur Encoding).

MaxSpur extends Spur by repurposing the *Character* tag (010) as a general *immediates* tag, allowing any class with a 5-bit index to be encoded as an immediate value. This means *True*, *False*, *UndefinedObject*, *Character*, *Symbol*, and certain block closures that capture no mutable state all avoid heap allocation. It also codes the *nil* object as all zeros - the same as a C null value.

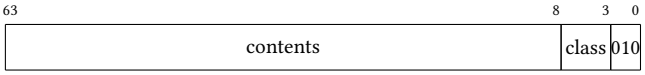
The default encoding for everything not described below remains encoded as the address:



The encoding for *SmallInteger* is:

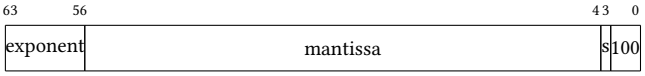


The encoding for immediate classes (including *Symbol*, *True*, *False*, *UndefinedObject*, *Character*, and some kinds of block closures) is:



where the `class` encodes up to 32 special classes in exactly the same way as Section ??.

The encoding for *ImmediateFloat* uses exactly the same encoding as *Spur*:



Only the pointer is used directly; all the others require some decoding before use.

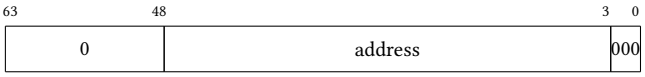
Values whose class index exceeds 5 bits, and floats outside the encodable exponent range, are heap-allocated.

2.2.7 Tag Most Possible Values (Max Encoding).

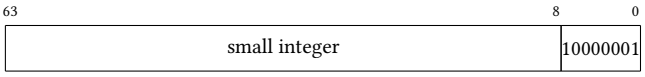
Our Smalltalk now uses a modified-Spur encoding. It is modified in three ways:

1. all 8 possible tag values used in the low 3 bits.
2. all non-float, non-heap-reference objects are encoded with an auxiliary tag.
3. the float encoding supports a wider exponent range and requires fewer instructions to decode and encode.

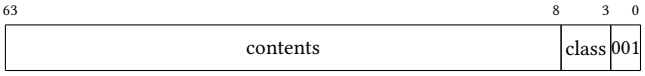
The default encoding for everything not described below (and the *nil* object, represented as null pointer; compatible with Zig/C) remains encoded as the address:



The encoding for 56-bit *SmallInteger* is:

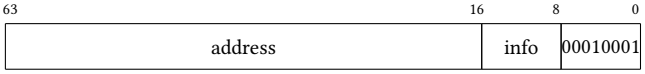


Which is a special case of the encoding for immediate classes (including *Symbol*, *True*, *False*, *UndefinedObject*, and *Character*), which is:



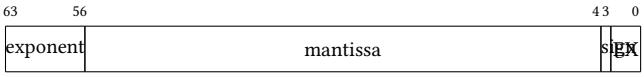
where the `class` encodes up to 32 special classes. There are several classes that encode special *BlockClosure* subclasses that are common in code.

For example, an immediate closure performing a non-local return of an instance variable is encoded as:



where `address` is the address of the *Context* to return from, and `info` is the 8-bit instance variable index within the *self* of that *Context*.

The *ImmediateFloat* encoding uses 6 of the 8 tag values, extending the Spur float range to all 64-bit values smaller than $\pm 2.68 \times 10^{154}$, including zero and small fractional values:



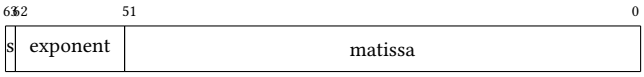
where `exponent` are the low 8 bits of the exponent, `mantissa` the mantissa bits, and `EX` the high 3 bits of the exponent offset by 2. This also reduces decode and encode cost by 2–3 instructions compared to Spur.

This increases the range of the representable floating point values to all 64-bit floats smaller than ± `2.68d154`, including 0 and the small fractional values. This encoding also saves a couple of instructions when decoding floating point values.

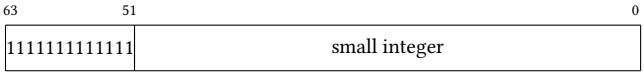
Examining the low 3 bits of an object disambiguates the encodings. Only the pointer tag is directly used; all the others require some decoding before use.

2.2.8 NaN Encoding.

The IEEE-754 64-bit floating point number is formatted as:



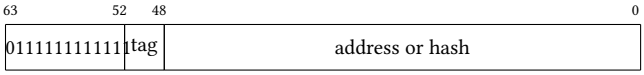
where if all the exponent bits are 1, the value is determined to be not-a-number (NaN). NaN encoding utilizes the large number of code points in the encoding that do not represent valid floating-point numbers, by using those bit patterns to represent values of other types, including pointers and *SmallInteger*. This encoding has been used by Spidermonkey[7], and was the originally planned encoding for our Smalltalk.There are various ways one might encode this, but for the purposes of this experiment we use the following. *SmallInteger* is encoded as:



To check for a *SmallInteger* we do an unsigned comparison, anything with the high 13 bits equal to 1 is a small integer, and the low 51 bits are a 51-bit two's-complement integer value.

The floating point numbers are treated as natural values, and only one value is designated as NaN - the value generated by the hardware. We recognize these by testing for NaN. If it's not a NaN then it's a floating point value. There is one false negative, so we check for that on a low-frequency path.

The rest of the values are:



where `tag` is a 4-bit tag: `0xF` encodes heap references; tags `1`–`9` encode immediate *BlockClosure* subclasses with varying parameter types; tags `A`–`D` encodes other immediate classes (symbols, characters, etc.).

This encoding recognizes all *SmallInteger* values with a single comparison and *Float* values with an additional comparison and floats require no conversion before use. All the other values, including references, require more bit manipulation to extract the class and value. Immediate values carry up to 48 bits of payload. As mentioned we can also encode some of the same *BlockClosure* subclasses values as described in Section 2.2.7, but with much-restricted parameters due to reduced available bit-width.

3 Experimental Design and Results

3.1 Experimental Design

Our Smalltalk was originally going to use NaN-encoding, and even after the decision was made to use the Max-encoding described in Section 2.2.7, the code for both was carried in parallel, although bit-rot set in for the NaN encoding as additional features were implemented. However the question of the best encoding has always remained.

There is a single enumeration in the runtime configuration module that determines which encoding will be used, so we can easily compile with any one of these encodings.

The experiment is to run the Fibonacci algorithms shown in Listing 1.

Listing 1. fibonacci

```
fibonacci1
        self <= 2 ifTrue: [ ^ self ].
        ^ (self - 1) fibonacci + (self -
            2) fibonacci
```

It will be run twice with *SmallInteger* and *Float* as receiver. We evaluate *40 fibonacci* or *40.0 fibonacci*. This will exercise the integer and floating-point paths.Note that for this particular benchmark, all of the floating point calculations are representable in all of the limited-range floating point encodings.

3.2 Results

Table 2 shows the total execution time of the various encodings to execute the benchmark (*40 fibonacci*) for integer and floating-point values on the Apple M1.

It also shows the timing for the native functions written in Zig. As it can be seen, there is significant overhead in our execution model that is used in the non-native executions. The *onlyInt* and *onlyFloat* use no encoding and no type-checking; everything is assumed to work correctly, so these are the fastest our model could possibly execute.[1]

[1]We are working on faster execution models, but that is irrelevant to this experiment.

Encoding	Native	NativeF	IntegerBr	Float
cachedPtr	198ms	300ms	3013ms	5039ms
nan	199ms	300ms	2999ms	2917ms
onlyFloat	198ms	300ms	N/A	2492ms
onlyInt	198ms	300ms	2622ms	N/A
ptr	199ms	300ms	4672ms	5136ms
spur	198ms	300ms	2674ms	4080ms
taggedHigh	199ms	300ms	2887ms	3558ms
taggedInt	198ms	300ms	2667ms	4678ms
taggedLow	198ms	300ms	2835ms	3859ms
max	198ms	300ms	2948ms	3086ms
maxSpur	199ms	300ms	2728ms	4187ms

Table 2. AArch64 (Apple M1) total timing for encodings

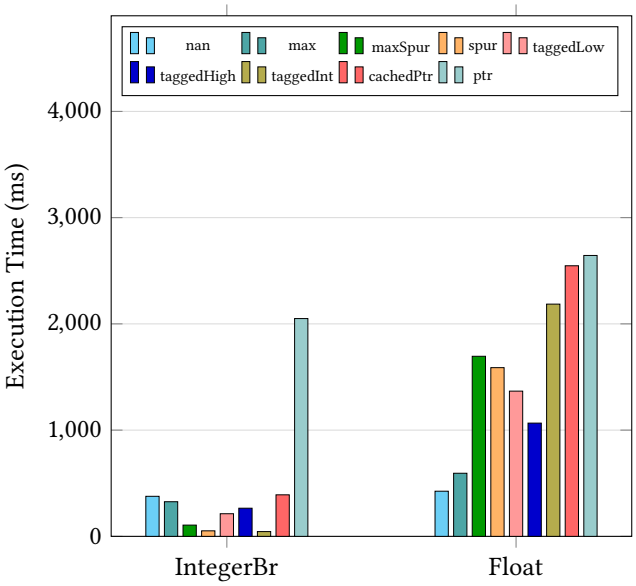


Figure 1. AArch64 benchmark results

To highlight the comparison among the encodings, in the rest of this paper we will use the net results - with the *onlyInt* or *onlyFloat* time subtracted - and omit the native times.[2]

Figures 1 and 2 show at a glance the relative performance of the various encodings on the two architectures.

All timings are the median of 10 runs following 3 runs of warmups. They are run on Linux systems running in console mode. We calculate the mean, median and geometric mean for each example and they are essentially equal in all cases.

Tables 3 and 4 present the net execution times for each encoding across the two benchmark configurations on AArch64 (Apple M1) and x86-64 respectively. Each row reports the time for a single encoding under two conditions: **IntegerBr**

[2]For reference the complete raw execution data is in the appendix.

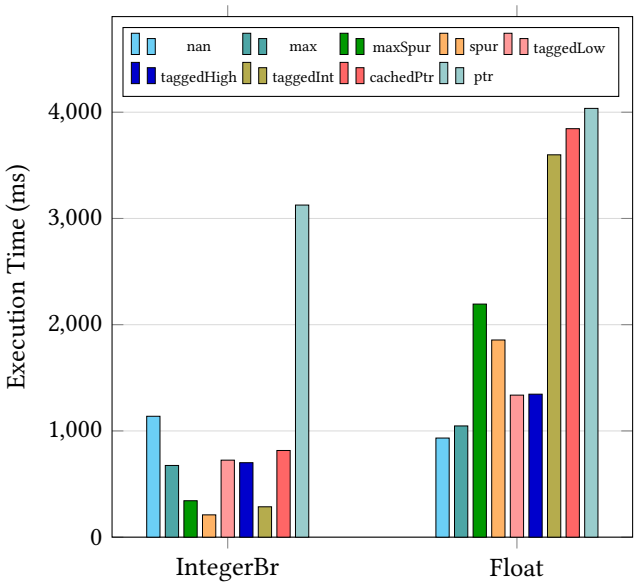


Figure 2. x86_64 benchmark results

(inlined branching integer Fibonacci), and **Float** (inlined float Fibonacci).

The AArch64 architecture seems to be better tuned for these encodings than x86-64 as all of the timings are faster - often significantly so, despite the x86-64 machine being about 20% faster based on the native executions.

3.2.1 Integer Performance (IntegerBr).

On AArch64, `taggedInt` dominates integer performance, followed by - `maxSpur` and `spur`. this ordering directly reflects tag-width economics, since `taggedInt` uses a single-bit tag, leaving 63 bits for the integer payloads. Extracting an immediate integer requires only a one-bit right shift which is a very inexpensive operation. `spur`'s three-bit tag adds marginal cost via wider shift, and `maxSpur` should be exactly the same, as they only differ in cases that are not exhibited in this benchmark, but they have slightly different execution times due to slightly different ordering of instructions because `maxSpur` handles more possible immediate values.

Similar results are visible on x86-64, where `taggedInt` remains a clear winner but the absolute gaps are wider. `nan` seems to degrade sharply between architectures on integer workloads, roughly tripling in cost on the x86-64 relative to Aarch64. *NaN-boxing* requires a bit-rotation and masking to recover the 47-bit integer payloads and x86-64 appears to penalize these operations more heavily than M1.

Not surprisingly, `ptr` encoding is slowest for integers for both architectures, roughly an order of magnitude slower than `taggedInt`. Every integer operation requires a heap

Encoding	IntegerBr	Float
nan	377ms	425ms
max	326ms	594ms
maxSpur	106ms	1695ms
spur	52ms	1588ms
taggedLow	213ms	1367ms
taggedHigh	265ms	1066ms
taggedInt	45ms	2186ms
cachedPtr	391ms	2547ms
ptr	2050ms	2644ms

Table 3. AArch64 (Apple M1) net timing for encodings

Encoding	IntegerBr	Float
nan	1138ms	933ms
max	675ms	1047ms
maxSpur	343ms	2194ms
spur	210ms	1856ms
taggedLow	725ms	1337ms
taggedHigh	701ms	1346ms
taggedInt	286ms	3599ms
cachedPtr	816ms	3845ms
ptr	3126ms	4036ms

Table 4. x86-64 (Linux) net timing for encodings

reference to read the value, polluting the cache with millions of small boxed objects.

3.2.2 Float Performance. *nan* encoding achieves the best float performance on both platforms. This is the fundamental advantage of *NaN-boxing*: IEEE 754 doubles are stored *unboxed* as the raw 64-bit representation, and type checking reduces to a simple *NaN* comparison with no memory indirection required for float operands.

max performs comparably, suggesting it also achieves efficient immediate float representation. *spur* offers a middle ground: its three-bit tag space accommodates both integer and float immediate tags, yielding float performance roughly three times slower than *nan* on Aarch64 but substantially faster than *taggedInt* (which allocates floats on the heap).

Float performance is thus dominated by where the encoding supports immediate floating-point values. Encodings that force heap allocation for floats pay 3-6x penalty relative to *nan* encoding.

3.2.3 Other observations. There are some interesting observations between the two architectures. Most notable is the relative difference between *taggedLow* and *taggedHigh*. On Aarch64, *taggedLow* is slightly faster for integer and *taggedHigh* is quite a bit faster for float. On x86-64 they are basically equivalent.

3.2.4 Pareto Analysis. The Pareto frontier contains the points that represent optimal trade-offs between two parameters. In this case, we are looking at integer versus floating-point performance in figure 3. The Pareto frontiers don't intersect, showing that the Aarch64 dominates the x86-64 in this experiment.

For Aarch64, many of the encodings are on the frontier:

- *nan* - best float performance, moderate integer cost;
- *max* - slightly slower float, slightly better integer;
- *taggedLow* and *taggedHigh* - strengthening integer, weakening float performance;
- *maxSpur* / *spur* - strong integer performance, moderate float cost;

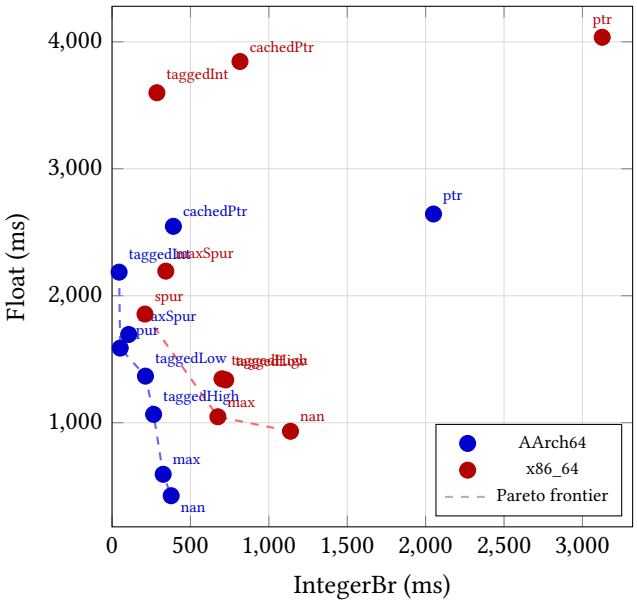


Figure 3. IntegerBr vs Float

- *taggedInt* - best integer performance, worst float performance on the frontier.

Encodings such as *ptr* are Pareto-dominated as they are outperformed in *both* dimensions by other encodings and can be ruled out for any modern production general-purpose workload. Their value lies primarily as experimental baselines that isolate the cost of heap-only object representations.

The *max* encoding occupies the most balanced position on the Pareto frontier, delivering competitive performance on both integer and float benchmarks.

For x86-64, only 3 encodings are on the frontier:

- *nan* - best float performance, moderate integer cost;
- *max* - only slightly slower float performance, but better integer, the most balanced position on the frontier;
- *spur* - very strong integer performance, significantly slower float.

4 Conclusions

There are few surprises coming out of this experiment: allocating objects in memory is slow, so encodings that need to allocate often are going to be slow.

Less obvious findings are that Nan-encoding is very good on AArch64, but a little slow on the integer workload on x86-64. The `max` encoding has a good balance on AArch64 and is the clear favourite on x86-64. There is no reason to use `spur` as `maxSpur` is a proper superset; they are faster than `max` for integer on AArch64 but slower on x86-64, and slower for floating-point on both platforms.

Floating-point values are naturally represented in `nan`, which could be useful for some kinds of foreign function work or GPU interaction. Pointers are naturally represented in `max`, `maxSpur`, and to a lesser extent `spur`, and this could be useful for foreign function interfacing.

Future Work

The Fibonacci benchmark, while exercising core dispatch and arithmetic paths, is a micro benchmark. Real Smalltalk applications exercise object allocation, polymorphic dispatch, string manipulation, and collection operations that may shift the performance balance between encodings.

In particular, encodings with wider tag spaces such as `max` with its three-bit tag and 16-bit class field may gain advantages in polymorphic dispatch that Fibonacci does not exercise.

The experiment should be extended with macro-benchmarks and application-level workloads to validate whether the rankings observed here hold under realistic conditions. In particular we look forward to validating our hypothesis that the extended range of immediate values that can be encoded in `max`, `nan`, `maxSpur`, and to a lesser extent `taggedHigh` and `taggedLow` will have significant performance advantages over `spur` and `taggedInt`.

References

[1] Clément Béra. 2018. 64 bits Immediate Floats. https://clementbera.wordpress.com/2018/11/09/64-bits-immediate-floats/. Blog post on the representation of immediate floating point numbers in the 64-bit Spur/OpenSmalltalk VM.

[2] L. Peter Deutsch and Allan M. Schiffman. 1984. Efficient implementation of the smalltalk-80 system. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Salt Lake City, Utah, USA) *(POPL '84)*. Association for Computing Machinery, New York, NY, USA, 297–302. doi:10.1145/800017.800542

[3] David A. Gudeman. 1993. Representing Type Information in Dynamically Typed Languages. *University of Arizona Technical Report* (1993). http://jimmyhmiller-bucket.s3.amazonaws.com/pdfs/programming-languages/6586051a27daa629dd94e06306cb3edad4f62dcf9579f65358e7940d8fed7ec0.pdf

[4] Olivier Melançon, Manuel Serrano, and Marc Feeley. 2025. Float Self-Tagging. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (Oct. 2025), 1620–1646. doi:10.1145/3763108

[5] Elliot Miranda. [n. d.]. *A Spur gear for Cog*. Retrieved 2022-06-01 from http://www.mirandabanda.org/cogblog/2013/09/05/a-spur-gear-for-cog/

[6] Eliot Miranda and Clément Béra. 2015. A Partial Read Barrier for Efficient Support of Live Object-oriented Programming. In *Proceedings of the 2015 International Symposium on Memory Management (ISMM)*. 93–104. doi:10.1145/2754169.2754186

[7] Mozilla. 2025. SpiderMonkey JavaScript Engine. https://spidermonkey.dev/ [Online; accessed 2025-04-08].

[8] Self Language Project. 2024. Self: Prototype-Based Programming Language and Environment. https://selflanguage.org/ Official website and documentation for the Self language.

[9] Guy L. Steele, Jr. 1977. *Data Representations in PDP-10 MacLISP*. Technical Report AI Memo 420. Massachusetts Institute of Technology, Artificial Intelligence Laboratory. https://dspace.mit.edu/handle/1721.1/6278

[10] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *Conference proceedings on Object-oriented programming systems, languages and applications*. 227–242.

A Appendix - Raw samples

AARCH64 raw execution

cachedPtr

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.30ms	0.1%	300ms
IntegerBr	3013ms	3012ms	0.54ms	0.0%	3012ms
Float	5039ms	5039ms	0.60ms	0.0%	5039ms

nan

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	199ms	199ms	0.00ms	0.0%	199ms
NativeF	300ms	300ms	0.00ms	0.0%	299ms
IntegerBr	2999ms	2998ms	0.75ms	0.0%	2998ms
Float	2917ms	2917ms	0.00ms	0.0%	2916ms

onlyFloat

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.00ms	0.0%	299ms
IntegerBr	N/A	N/A	N/A	N/A	N/A
Float	2492ms	2490ms	2.57ms	0.1%	2490ms

onlyInt

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.30ms	0.1%	300ms
IntegerBr	2622ms	2622ms	7.30ms	0.3%	2622ms
Float	N/A	N/A	N/A	N/A	N/A

ptr

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	199ms	198ms	0.40ms	0.2%	198ms
NativeF	300ms	300ms	0.00ms	0.0%	299ms
IntegerBr	4672ms	4672ms	0.30ms	0.0%	4672ms
Float	5136ms	5136ms	1.28ms	0.0%	5136ms

spur

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.00ms	0.0%	299ms
IntegerBr	2674ms	2674ms	0.49ms	0.0%	2674ms
Float	4080ms	4080ms	0.64ms	0.0%	4080ms

taggedHigh

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	199ms	199ms	0.00ms	0.0%	199ms
NativeF	300ms	300ms	0.30ms	0.1%	300ms
IntegerBr	2887ms	2887ms	0.00ms	0.0%	2887ms
Float	3558ms	3558ms	0.30ms	0.0%	3558ms

taggedInt

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.40ms	0.1%	300ms
IntegerBr	2667ms	2667ms	0.00ms	0.0%	2667ms
Float	4678ms	4678ms	0.30ms	0.0%	4678ms

taggedLow

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.46ms	0.2%	300ms
IntegerBr	2835ms	2834ms	0.50ms	0.0%	2834ms
Float	3859ms	3859ms	0.60ms	0.0%	3859ms

max

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	198ms	198ms	0.00ms	0.0%	198ms
NativeF	300ms	300ms	0.00ms	0.0%	299ms
IntegerBr	2948ms	2947ms	1.20ms	0.0%	2947ms
Float	3086ms	3086ms	0.40ms	0.0%	3086ms

maxSpur

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	199ms	198ms	0.46ms	0.2%	198ms
NativeF	300ms	300ms	0.30ms	0.1%	300ms
IntegerBr	2728ms	2728ms	0.45ms	0.0%	2727ms
Float	4187ms	4187ms	0.77ms	0.0%	4186ms

X86-64 raw execution

cachedPtr

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	134ms	133ms	0.46ms	0.3%	133ms
NativeF	283ms	282ms	0.66ms	0.2%	282ms
IntegerBr	4779ms	4778ms	3.27ms	0.1%	4778ms
Float	7673ms	7672ms	2.15ms	0.0%	7672ms

nan

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	168ms	167ms	0.46ms	0.3%	167ms
NativeF	245ms	245ms	0.49ms	0.2%	245ms
IntegerBr	5101ms	5099ms	2.91ms	0.1%	5099ms
Float	4761ms	4761ms	2.72ms	0.1%	4761ms

onlyFloat

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	132ms	132ms	0.81ms	0.6%	132ms
NativeF	282ms	282ms	0.30ms	0.1%	282ms
IntegerBr	N/A	N/A	N/A	N/A	N/A
Float	3828ms	3825ms	3.46ms	0.1%	3825ms

onlyInt

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	134ms	134ms	0.40ms	0.3%	134ms
NativeF	282ms	282ms	0.40ms	0.1%	282ms
IntegerBr	3963ms	3962ms	0.54ms	0.0%	3962ms
Float	N/A	N/A	N/A	N/A	N/A

ptr

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	168ms	168ms	0.30ms	0.2%	168ms
NativeF	245ms	245ms	0.49ms	0.2%	245ms
IntegerBr	7089ms	7089ms	0.75ms	0.0%	7089ms
Float	7864ms	7863ms	1.57ms	0.0%	7863ms

spur

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	134ms	133ms	0.30ms	0.2%	133ms
NativeF	282ms	282ms	0.40ms	0.1%	282ms
IntegerBr	4173ms	4173ms	2.79ms	0.1%	4173ms
Float	5684ms	5684ms	1.78ms	0.0%	5684ms

taggedHigh

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	145ms	145ms	0.00ms	0.0%	144ms
NativeF	242ms	241ms	0.50ms	0.2%	241ms
IntegerBr	4664ms	4661ms	15.83ms	0.3%	4661ms
Float	5174ms	5173ms	2.24ms	0.0%	5173ms

taggedInt

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	164ms	164ms	0.46ms	0.3%	164ms
NativeF	236ms	236ms	0.49ms	0.2%	236ms
IntegerBr	4249ms	4249ms	1.34ms	0.0%	4248ms
Float	7427ms	7426ms	1.02ms	0.0%	7426ms

taggedLow

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	133ms	133ms	0.45ms	0.3%	132ms
NativeF	282ms	282ms	0.40ms	0.1%	282ms
IntegerBr	4688ms	4688ms	3.11ms	0.1%	4688ms
Float	5165ms	5165ms	0.63ms	0.0%	5164ms

max

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	145ms	145ms	0.00ms	0.0%	144ms
NativeF	243ms	242ms	0.50ms	0.2%	242ms
IntegerBr	4638ms	4637ms	0.94ms	0.0%	4637ms
Float	4875ms	4877ms	6.69ms	0.1%	4877ms

maxSpur

Benchmark	Median	Mean	StdDev	SD%	GeoM
Native	168ms	168ms	0.45ms	0.3%	167ms
NativeF	245ms	245ms	0.49ms	0.2%	245ms
IntegerBr	4306ms	4305ms	1.04ms	0.0%	4305ms
Float	6022ms	6021ms	3.03ms	0.1%	6021ms